

Молдавский Государственный Университет
Факультет Прикладной информатики
Министерство образования и исследования.
Очное отделение

Отчет по лабораторной работе№3

Выполнил: студент группы IA2504:Зебели Василий

Кишинев 2026

Тема: Работа с массивами и функциями в JavaScript для анализа транзакций

Цель работы

Изучить основы работы с массивами и функциями в JavaScript, применяя их для обработки и анализа транзакций.

Ход работы

1. Создание массива транзакций

В файле `main.js` был создан массив объектов `transactions`, содержащий информацию о транзакциях.

Каждая транзакция включает следующие свойства:

- `transaction_id` — уникальный идентификатор транзакции;
- `transaction_date` — дата транзакции;
- `transaction_amount` — сумма транзакции;
- `transaction_type` — тип транзакции (`debit` или `credit`);
- `transaction_description` — описание транзакции;
- `merchant_name` — название магазина или сервиса;
- `card_type` — тип карты.

Было создано несколько тестовых транзакций для проверки работы функций.

2. Реализация функций

В ходе работы были реализованы следующие функции:

2.1 *`getUniqueTransactionTypes(transactions)`*

Функция возвращает массив уникальных типов транзакций.

Для выполнения использовались:

- `map()` — для получения типов транзакций;
- `Set()` — для удаления повторяющихся значений.

```

32
33 console.log(transactions);
34
35 function getUniqueTransactionTypes(transactions) {
36
37     const types = transactions.map(
38         transaction => transaction.transaction_type
39     );
40
41     return [...new Set(types)];
42 }
43
44 console.log(
45     getUniqueTransactionTypes(transactions)
46 );

```

Результат работы функции:

```

transaction_id: 5,
transaction_date: '2024-04-10',
transaction_amount: 150,
transaction_type: 'debit',
transaction_description: 'Кафе',
merchant_name: 'Coffee Shop',
card_type: 'debit'
}
]
[ 'debit', 'credit' ]

```

2.2 *calculateTotalAmount(transactions)*

Функция вычисляет общую сумму всех транзакций.

Для выполнения использовался метод:

- `reduce()` — для последовательного суммирования значений.

```

44 function calculateTotalAmount(transactions) {
45
46     return transactions.reduce(
47         (sum, transaction) =>
48             sum + transaction.transaction_amount,
49         0
50     );
51 }

```

Результат:

```

    transaction_description: 'Кафе',
    merchant_name: 'Coffee Shop',
    card_type: 'debit'
  }
]
[ 'debit', 'credit' ]
1400
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js>

```

2.3 *getTransactionByType(transactions, type)*

Функция возвращает транзакции определённого типа (debit или credit).

Для выполнения использовался:

- `filter()` — для фильтрации массива.

```

53 function getTransactionByType(
54     transactions,
55     type
56 ) {
57
58     return transactions.filter(
59         transaction =>
60             transaction.transaction_type === type
61     );
62 }

```

Результат:

```

    transaction_id: 3,
    transaction_date: '2024-04-10',
    transaction_amount: 150,
    transaction_type: 'debit',
    transaction_description: 'Кафе',
    merchant_name: 'Coffee Shop',
    card_type: 'debit'
  }
]
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js>

```

2.4 *calculateAverageTransactionAmount(transactions)*

Функция вычисляет среднее значение всех транзакций.

Выполняется:

1. вычисление общей суммы транзакций;
2. деление суммы на количество элементов массива.

```
63 function calculateAverageTransactionAmount(transactions) {  
64  
65     if (transactions.length === 0) {  
66         return 0;  
67     }  
68  
69     const total =  
70         calculateTotalAmount(transactions);  
71  
72     return total / transactions.length;  
73 }
```

Результат:

```
transaction_description: 'Кафе',  
merchant_name: 'Coffee Shop',  
card_type: 'debit'  
}  
]  
466.6666666666667  
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js>
```

2.5 *getTransactionsByMerchant(transactions, merchantName)*

Функция возвращает транзакции, совершённые у определённого продавца (merchant_name).

Для поиска использовался метод:

- `filter()`.

```

75  function getTransactionsByMerchant(
76      transactions,
77      merchantName
78  ) {
79
80      return transactions.filter(
81          transaction =>
82              transaction.merchant_name === merchantName
83      );
84  }

```

Результат:

```

    transaction_id: 1,
    transaction_date: '2024-03-15',
    transaction_amount: 250,
    transaction_type: 'debit',
    transaction_description: 'Покупка продуктов',
    merchant_name: 'Linella',
    card_type: 'debit'
  }
]
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js>

```

getTransactionsByAmountRange(transactions, minAmount, maxAmount)

Функция возвращает транзакции в заданном диапазоне сумм.

```

120
127  console.log(
128      getTransactionsByAmountRange(
129          transactions,
130          100,
131          500
132      )
133  );

```

В результате были найдены соответствующие транзакции:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
transaction_date: '2024-03-15',
transaction_amount: 250,
transaction_type: 'debit',
transaction_description: 'Покупка продуктов',
merchant_name: 'Linella',
card_type: 'debit'
},
{
  transaction_id: 3,
  transaction_date: '2024-04-10',
  transaction_amount: 150,
  transaction_type: 'debit',
  transaction_description: 'Кафе',
```

findTransactionById(transactions, id)

Функция выполняет поиск транзакции по уникальному идентификатору.

Для выполнения использовался метод:

- `find()`.

```
146 console.log(
147   findTransactionById(
148     transactions,
149     2
150   )
151 );
```

Была успешно найдена транзакция с указанным ID:

```
{
  transaction_id: 2,
  transaction_date: '2024-03-20',
  transaction_amount: 1000,
  transaction_type: 'credit',
  transaction_description: 'Зарплата',
  merchant_name: 'Company',
  card_type: 'credit'
}
PS C:\Users\Vasili\OneDrive\Desktop\1 курс\js>
```

В ходе выполнения лабораторной работы были использованы следующие методы:

- `map()` — преобразование массива;
- `filter()` — фильтрация данных;
- `reduce()` — вычисление суммы;

- `find()` — поиск элемента;
- `Set()` — получение уникальных значений.

Тестирование

Все реализованные функции были протестированы с использованием `console.log()`.

Проверка показала корректную работу функций и правильный вывод результатов в консоль.

Контрольные вопросы

- 1. Какие методы массивов можно использовать для обработки объектов в JavaScript?**
2. Для обработки объектов в массивах JavaScript используют методы: `map()`, `filter()`, `find()`, `findIndex()`, `reduce()`, `sort()`, `forEach()`, `some()`, `every()` и другие. Они позволяют изменять, фильтровать, искать, сортировать и анализировать данные.
- 3. Как сравнивать даты в строковом формате в JavaScript?**

Даты в строковом формате обычно сравнивают после преобразования в объект `Date`. Если дата записана в формате `YYYY-MM-DD`, её можно сравнивать и как обычную строку, так как порядок символов соответствует хронологическому порядку.

- 4. В чём разница между `map()`, `filter()` и `reduce()` при работе с массивами объектов?**

`map()` используется для преобразования каждого элемента массива и создания нового массива. `filter()` применяется для отбора элементов, которые соответствуют условию. `reduce()` нужен для объединения всех элементов массива в одно итоговое значение, например сумму, объект или строку.

Вывод

В ходе выполнения лабораторной работы были изучены основы работы с массивами и функциями в JavaScript. Были реализованы функции для анализа транзакций: поиск уникальных типов операций, вычисление общей и средней суммы, фильтрация по типу транзакции, магазину и диапазону сумм, а также поиск транзакции по идентификатору. Получены практические навыки использования методов `map()`, `filter()`, `reduce()` и `find()` для обработки массивов объектов.